# Shiam Ezzo

## Owner onboarding and subscription backend

Branch: Sheam | Plan begins when the team receives the starter.

Build one focused owner module: pharmacies, pharmacist accounts and subscription management. Build your owner module and subscription helper yourself. Agree the schema and authentication interfaces with Mustafa; authentication is his separate responsibility.

## Your code ownership

- apps/api/src/modules/owner/**

- apps/api/src/common/subscription-status.ts

- apps/api/src/common/subscription-status.spec.ts

## Your agreed interfaces

- GET/POST /owner/pharmacies; POST /owner/pharmacists

- POST/PATCH/DELETE /owner/pharmacies/:id/subscription

- Implement/test effectiveSubscriptionStatus(subscription,today); agree this reusable interface with other API owners.

- All routes OWNER-only; return OwnerPharmacy/Subscription/SessionUser shapes from contracts.

## Start without waiting

Pull the foundation from main into your branch. Run npm ci, copy both apps/api/.env.example and apps/web/.env.example to .env on first setup, then npm run dev. The frontend shows only a plain startup message. No database models/migrations/seed or product features are implemented. Mustafa authors the database setup; follow his instructions after that PR merges.

Read the final specification, proposed API contract, acceptance checklist and your task folders. You build the assigned pages/services yourself. There are no prebuilt guards, calculations, forms, components, adapters or fixtures. Agree interfaces early and create your own small local test data to work independently.

## Scope boundary

No frontend, login implementation, inventory edits, billing or schema migrations. Request schema changes from Mustafa. The existing Git branch is spelled Sheam even though the task guide uses Shiam.

# Week 1 - build your area

Days are relative, not the instructor deadline. Day 7 is a review/buffer day; adjust to your availability. Deliver small reviewed PRs throughout the week.

### Day 1

Run the empty backend. Agree Pharmacy/User/Subscription fields with Mustafa, then plan your own owner controller/service/DTOs and subscription-status helper. Read the exact status rules in the teacher specification.

### Day 2

Write effectiveSubscriptionStatus and its tests from scratch: null, future start, inclusive boundaries, expired and suspended. Share its interface/PR early. Use your own fake Prisma fixtures while authentication is in progress.

### Day 3

Implement create pharmacy and list/search pharmacies. Return effective subscription status using one captured business date; add DTO validation and explicit response mapping.

### Day 4

Implement pharmacist account creation linked to an existing pharmacy. Normalize email, hash initial password, reject duplicates and never return/log the hash.

### Day 5

Implement create subscription: one per pharmacy, strict date validation, end after start. Test conflict for a second subscription; use the database unique constraint.

### Day 6

Implement subscription date edits, suspend/reactivate and deletion. Delete only Subscription; all stock/accounts/duty/history must remain. Log safe business events.

### Day 7

Buffer/review: attach Mustafa's guards, add owner-only Swagger docs and status filters. Open a complete owner API PR for Week 1 review.

# Week 2 - integrate and prove

Day 8: API integration. Day 12: core acceptance. Days 13-14: English rehearsal and fixes. No new optional features while CORE is failing.

### Day 8

Connect the real owner endpoints with Alaa's UI. Verify all dates, statuses and validation messages match the contracts.

### Day 9

Add controller/API tests: anonymous fails 401, pharmacist fails 403, nonexistent pharmacy/subscription fails 404 and invalid UUID/body fails 400.

### Day 10

Run the subscription lifecycle: create ACTIVE, suspend, reactivate, expire through test clock, delete to NONE. Ensure filtering/search uses effective rather than stored status.

### Day 11

Pair briefly with Hamid and Mustafa: changing/deleting subscription must remove public listing and block pharmacist writes while preserving reads/data.

### Day 12

Check dates in Asia/Amman around midnight; add regression tests for future-start NONE and suspended-but-expired precedence. Fix Swagger/example/error inconsistencies.

### Day 13

Full-flow rehearsal with seeded owner and two pharmacist accounts. Confirm no password/hash in any owner response. Prepare English demo steps.

### Day 14

Buffer/final fixes. Show owner onboarding and subscription deletion preserving pharmacy data. Explain ACTIVE/EXPIRED/SUSPENDED/NONE in English.

# Ready-to-merge checklist

[ ] A03: owner creates pharmacy and pharmacist linked to it; email conflict/nonexistent pharmacy rejected.

[ ] A04: inclusive date boundaries, suspension precedence, future/missing NONE; one subscription only; end>start.

[ ] A06: deleting subscription retains users/pharmacy/medicines/batches/duty exactly.

[ ] A01/A05: only owner changes subscriptions; inactive pharmacist read/write/public behavior integrates correctly.

[ ] Unknown fields rejected; proper HTTP errors; summaries/parameters/auth/responses in Swagger; safe logs.

## Handoffs and dependencies

- Independent start: write/test your status helper and service using your own fake data. Agree database fields and wire types on Day 1.

- Day 2-3: use Mustafa's shared guards; ask him for review of account creation/security.

- Day 5: give Alaa sample owner payloads; Day 7: owner endpoints available.

- Day 11: verify subscription effects with Hamid/Mustafa without editing their modules.

## Every pull request

Commit only your intended files; push your branch; open a PR into main. Include the task/rule IDs, a short explanation, and tests or screenshots. Run npm run typecheck, npm test and npm run build. Backend contributors also document their endpoints in Swagger. Alaa reviews/merges; keep .env, node_modules, dist and temporary files untracked.

## Your English presentation segment

Create a pharmacy/account, change its subscription, delete that subscription and show that inventory still exists. Explain the difference between stored suspension and calculated effective status.

Everyone should understand the whole flow: owner onboarding -> pharmacist stock -> dashboard -> public search. Be ready to explain a rule outside your own feature. Final acceptance uses real API requests and persisted PostgreSQL data; locally authored test fixtures are only for development.